

Reinforcement Learning

Introduction

Textbook Chapter 1

Michael Hahsler, SMU

Based on slides by Oliver Wallscheid (Paderborn University) and material by David Silver (University College London)



Topics of this course

- Introduction to reinforcement learning
- Markov decision processes
- Dynamic programming
- Monte Carlo methods
- Temporal-difference learning
- Multi-step bootstrapping
- Planning and learning with tabular methods
- Function approximation with supervised learning
- On-policy prediction with function approximation
- Value-based control with function approximation
- Stochastic policy gradient methods
- Deterministic policy gradient methods

Recommended textbooks

- Reinforcement learning: an introduction,
R. Sutton and G. Barto
MIT Press, 2nd edition, 2020
Available [here](#)
- Reinforcement learning (lecture script)
D. Silver
Entire slide set available [here](#)
YouTube lecture series (click [here](#))

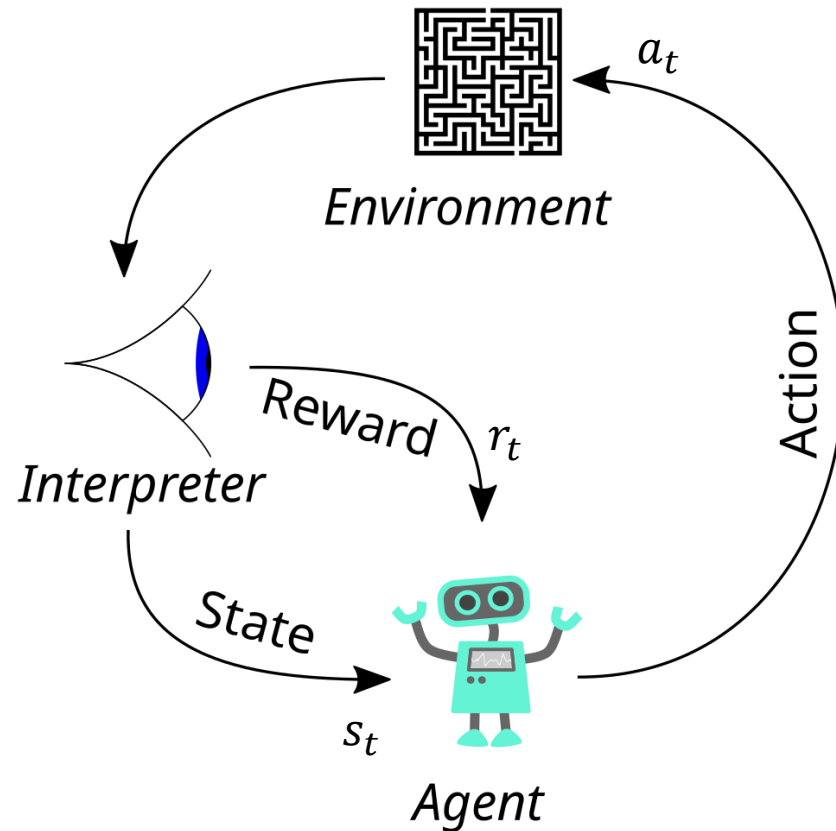
Table of contents

- What is RL?
- RL basics
 - Reward
 - States
 - Actions and policies
- A small example
- RL and planning

A hand is holding a yellow, bone-shaped treat in the upper left corner. A white dog with brown patches on its head and ears is lying down, looking up at the treat. The dog is wearing a blue collar with a red stripe. The background is a solid light gray.

What is RL?

The basic reinforcement learning structure



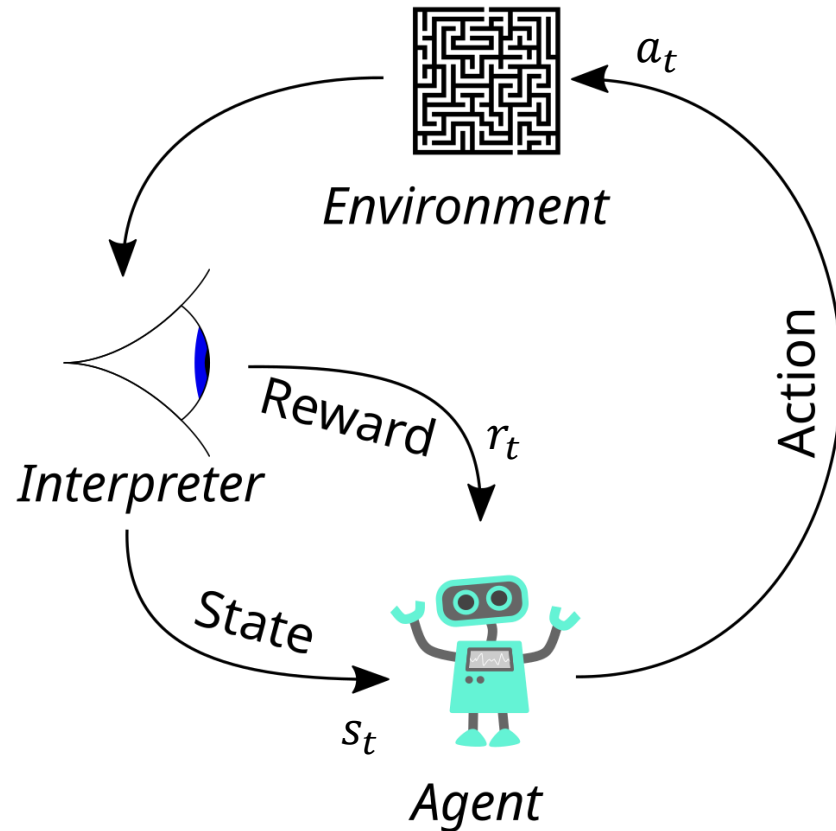
The basic RL operation principle
(derivative of www.wikipedia.org, CC0 1.0)

Goal: Map a situation to an action to maximize the reward

Key characteristics:

- **Uncertainty** about the environment leads to a need for sensing.
- **No supervisor:** discover actions by trying them (data-driven)
- **Sequential decision making:** Agent actions affect subsequent data. Agent maximizes over future (delayed) rewards which need to be estimated!
- Action discovery and maximization lead to a tradeoff: **exploit vs. explore**

Agent and environment



At each step t the agent:

- Picks an action a_t .
- Receives the state s_t .
- Receives a reward r_t .

At each step t the environment:

- Receives an action a_t .
- Emits an observation s_{t+1} .
- Emits a reward r_{t+1} .

The time increments $t \leftarrow t + 1$.

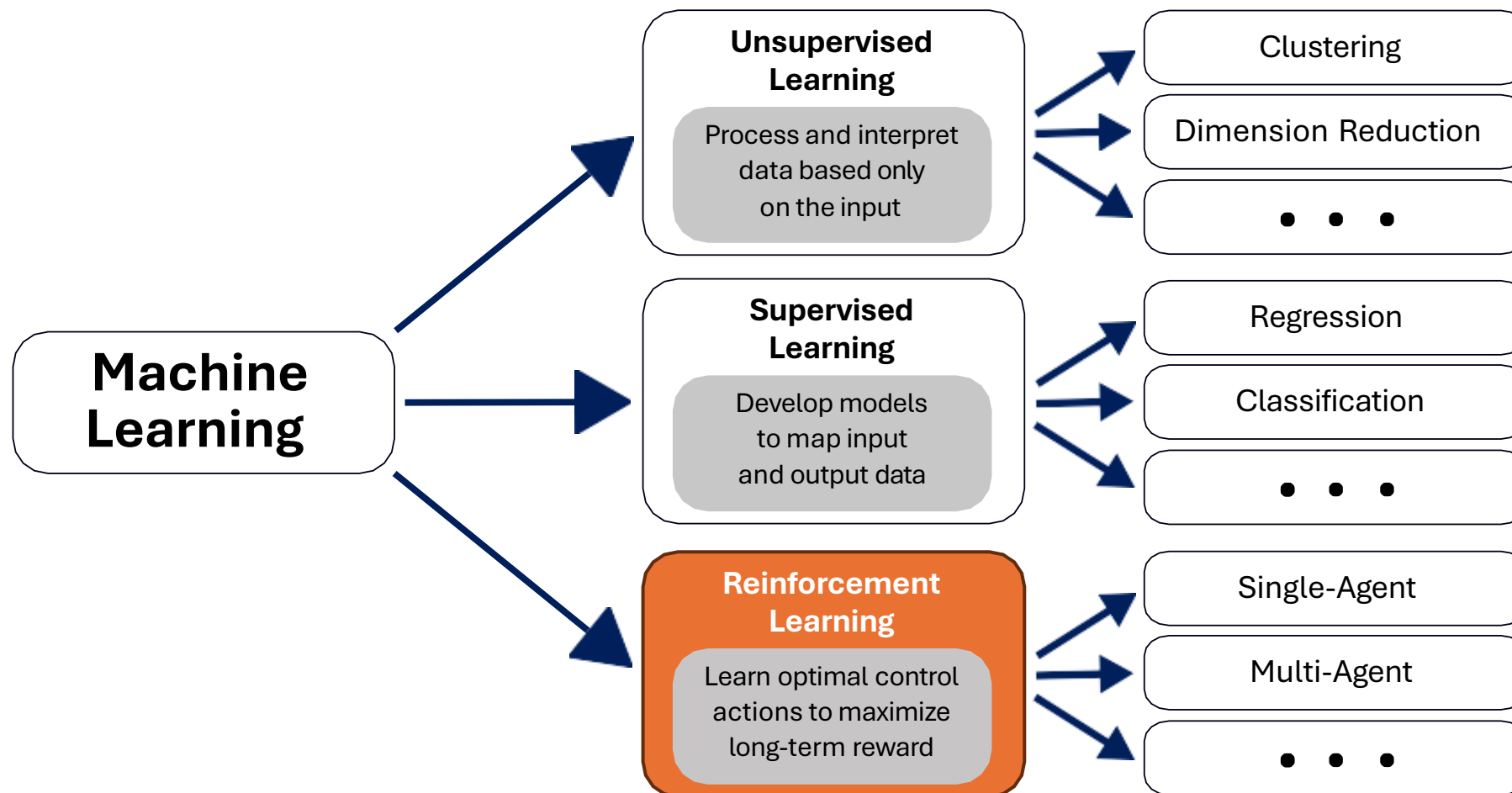
Remark on time: A one step time delay is assumed between executing the action and receiving the observation as well as reward.

Some basic definitions from the literature

- What is reinforcement?
 - “Reinforcement refers to consequences that increase the likelihood of an organism’s future behavior, typically in the presence of a particular antecedent stimulus.[...] Reinforcers serve to increase behaviors whereas punishers serve to decrease behaviors; thus, positive reinforcers are stimuli that the subject will work to attain, and negative reinforcers are stimuli that the subject will work to be rid of or to end.”, source: [wikipedia.org](https://en.wikipedia.org) (obtained 2025-03-19)
- What is learning?
 - “Acquiring knowledge and skills and having them readily available from memory so you can make sense of future problems and opportunities.”, source: Make It Stick: The Science of Successful Learning, Brown et al., Harvard Press, 2014

RL and machine learning

RL is a 3rd type of machine learning



Context around machine learning

Deep Learning (DL)

A class of ML which uses large, layered models (e.g., vast artificial neural networks) to progressively extract more information from the data.

Machine Learning (ML)

A subset of AI involved with the creation of algorithms which can modify itself without human intervention to produce desired output by feeding itself through structured data.

Artificial Intelligence (AI)

Any device that perceives its environment and takes actions that maximize its chance of successfully achieving its goals. AI is often used to describe machines that mimic "cognitive" functions that humans associate with the human mind.

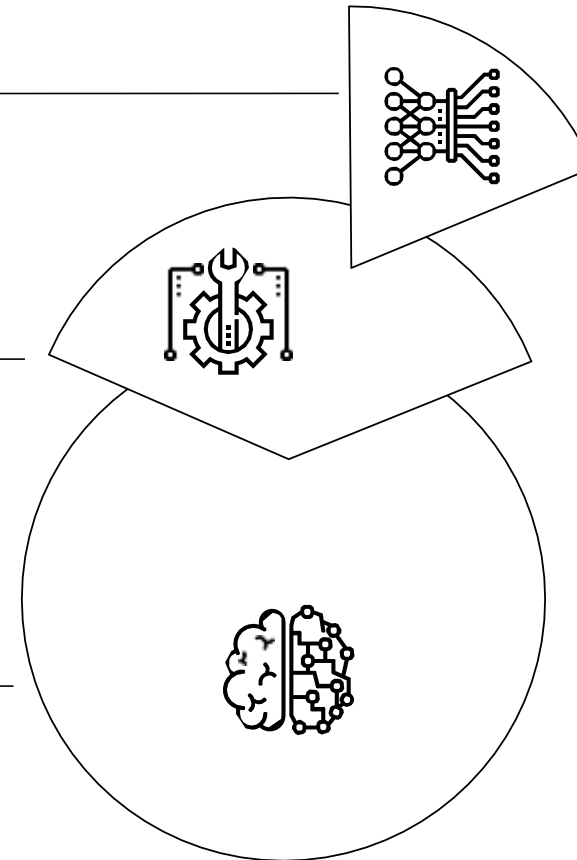
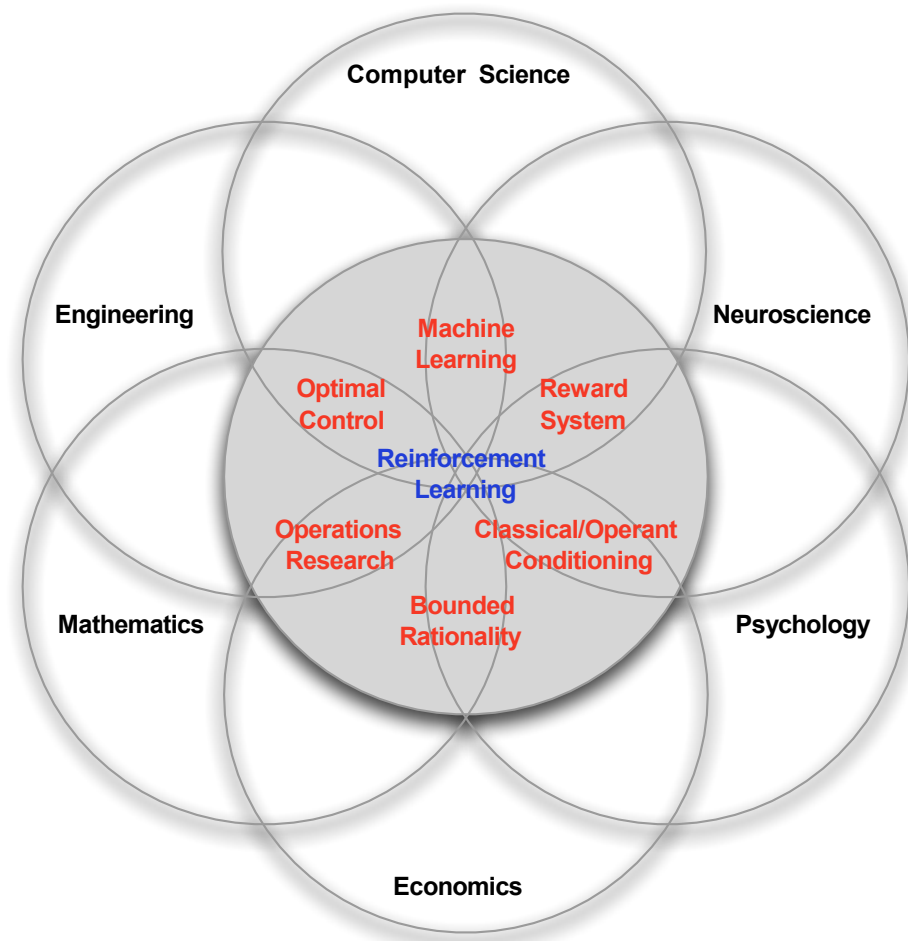


Figure 1.3: The broader scope around machine learning

Many faces of reinforcement learning

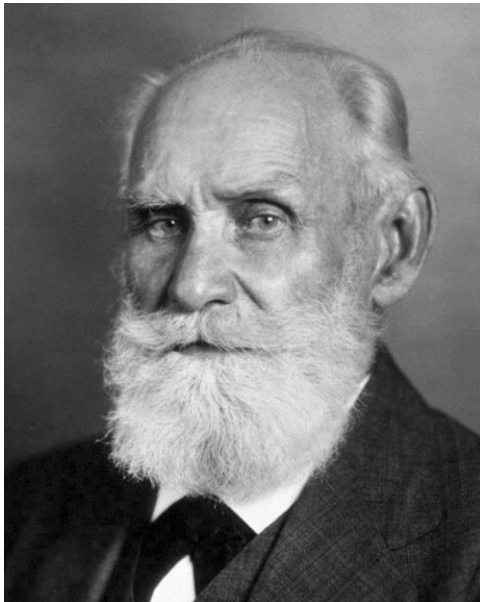


RL and its neighboring domains

(source: D. Silver, “Reinforcement learning”, 2015. [CC BY-NC 4.0](https://creativecommons.org/licenses/by-nc/4.0/))

Methodical origins

Classical conditioning



Ivan Pavlov (1849-1936)
Russian physiologist and Nobel laureate who discovered classical conditioning through his experiments with dogs.

Stochastic process formalism



Andrei Markov (1856-1922)
Russian mathematician developed the groundwork for Markov chains to model dynamical systems.

Optimal sequential decision making



Richard E. Bellman (1920-1984)
American applied mathematician, who introduced dynamic programming and the Bellman equation.

History of reinforcement learning

Huge field with many interconnections to different fields. One could give a lecture only on the historic development. Hence, interested readers are referred to:

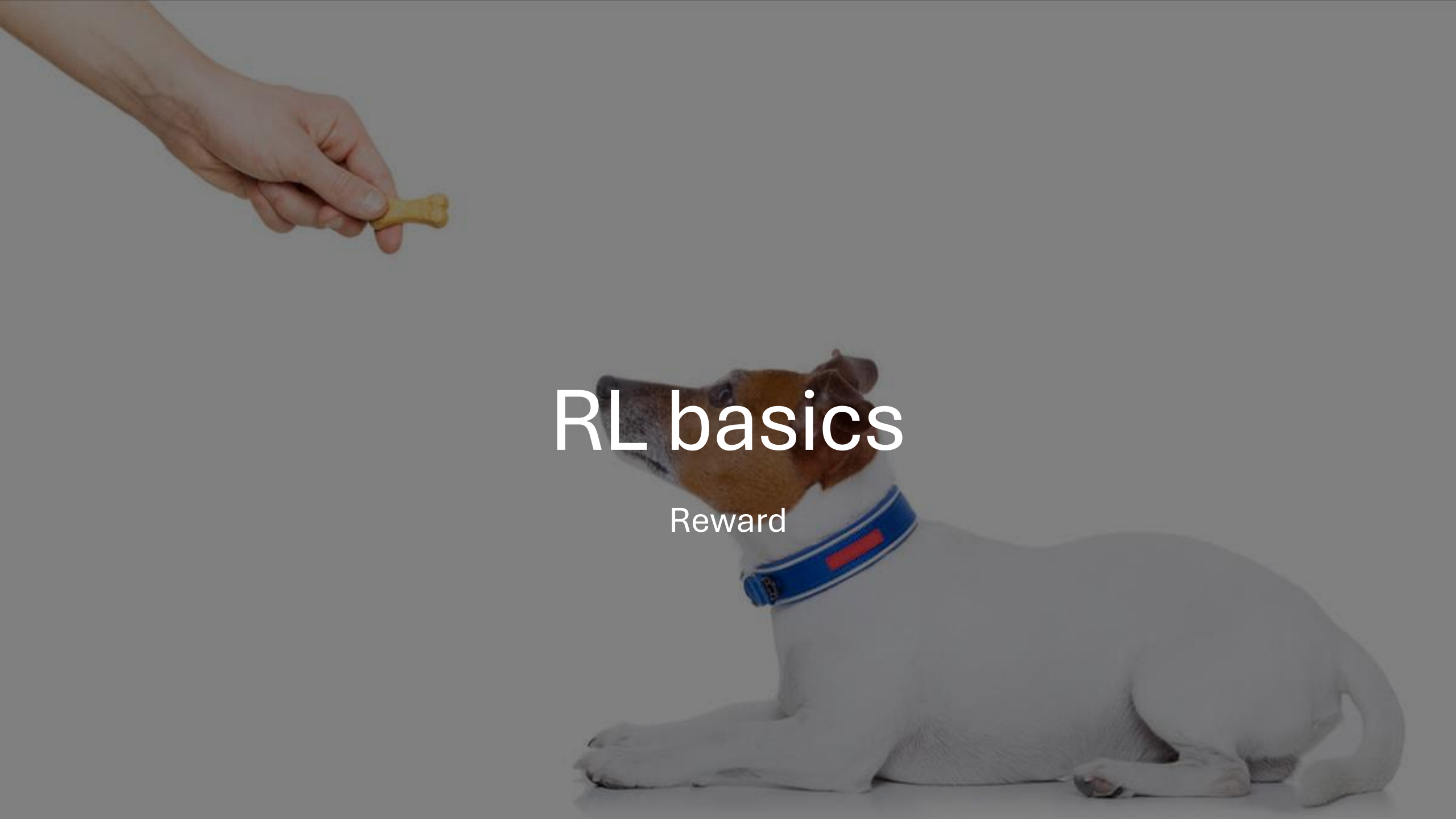
- Chapter 1.7 of Barto/Sutton, Reinforcement learning: an introduction, 2nd edition, MIT Press, 2018
- 30 minutes talk of A. Barto:
<https://www.youtube.com/watch?v=ul6B2oFPNDM>
- Survey papers on historic as well as more recent developments:
 - Kaelbling et al., [Reinforcement learning: A survey](#), in Journal of Artificial Intelligence Research, vol. 4, pp. 237 - 285, 1996
 - Arulkumaran et al., [Deep reinforcement learning: a brief survey](#), in IEEE Signal Processing Magazine, vol. 34, no. 6, pp. 26-38, 2017
 - Botvinick et al., [Reinforcement learning, fast and slow](#), in Trends in Cognitive Sciences, vol. 23, iss. 5, pp. 408-422, 2019

Contemporary application examples

- Controlling electric drive systems
- Swinging-up and balance a cart-pole / an inverted pendulum
- Flipping pancakes with a roboter arm
- Drifting with a RC-car
- Driving an autonomous car
- Playing Atari Breakout
- Play strategy board game Go at super-human performance
- Nuclear fusion reactor plasma control
- Training chat bots (like chatGPT)
- ...

RL basics

Reward



Notation

Capital letters random variables
Lower case letters realizations of random variables and for scalar functions
Bold lower-case letters real-valued vectors (even if random variables)
Bold capitals matrices

$\Pr\{X = x\}$ probability that a random variable X takes on the value x
 $X \sim p$ random variable X selected from distribution $p(x) = \Pr\{X = x\}$
 $\mathbb{E}[X]$ expectation of a random variable X , i.e., $\mathbb{E}[X] = \sum_x p(x)x$
 $\operatorname{argmax}_a f(a)$ a value of action a at which $f(a)$ takes its maximal value
 γ discount-rate parameter

s, s' states
 a an action
 r a reward
 $\mathcal{S}$ set of all (nonterminal) states
 $\mathcal{A}(s)$ set of all actions available in state s
 t discrete time step
 T final time step of an episode (a.k.a. horizon)
 A_t random variable for the action at time t
 S_t random variable for the state at time t
 R_t random variable for the reward at time t
 π policy (decision-making rule)
 $\pi(s)$ action taken in state s under deterministic policy π
 $\pi(a|s)$ probability of taking action a in state s under stochastic policy π

G_t return (cumulative reward) following time t
 $p(s' | s, a)$ probability of transition to state s' , from state s taking action a
 expected immediate reward from state s after action a
 $v_\pi(s)$ value of state s under policy π (expected return)
 $v_*(s)$ value of state s under the optimal policy
 $q_\pi(s, a)$ value of taking action a in state s under policy π
 $q_*(s, a)$ value of taking action a in state s under the optimal policy

Reward

- An immediate reward is a scalar random variable R_t with realizations r_t that the agent gets at time t .
- Often it is considered a real-number $r_t \in \mathbb{R}$ or an integer $r_t \in \mathbb{Z}$.
- The reward function may be naturally given or is decided on by the designer (i.e., can be manipulated).
- It fully indicates how well an RL agent is doing at step t .
- The agent's task is to maximize its reward over time.

Reward hypothesis: All goals can be described by the maximization of the expected cumulative reward:

$$\max \mathbb{E} \left[\sum_{t=0}^{\infty} R_t \right]$$

Reward examples

- Flipping a pancake:
 - Pos. reward: catching the 180° rotated pancake
 - Neg. reward: dropping the pancake on the floor
- Stock trading:
 - Trading portfolio monetary value
- Playing Atari games:
 - Highscore value at the end of a game episode
- Driving an autonomous car:
 - Pos. reward: getting save from A to B without crashing
 - Neg. reward: hitting another car, pedestrian, bicycle,...
- Classical control task (e.g., electric drive, inverted pendulum,...):
 - Pos. reward: following a given reference trajectory precisely
 - Neg. reward: violating system constraints and/or large control error

Reward characteristics

Rewards are highly depending on the given problem:

- Actions may have short and/or long-term consequences.
 - The reward for a certain action may be delayed.
 - Examples: Stock trading, strategic board games,...
- Rewards can be positive and negative values.
 - Certain situations (e.g., car hits wall) might lead to a negative reward (= a cost).
- Exogenous impacts might introduce stochastic reward components.
 - Example: A wind gust pushes an autonomous helicopter into a tree.

Remarks on reward

The RL agent's learning process is heavily linked with the reward distribution over time. Designing expedient rewards functions is therefore crucially important for successfully applying RL. And often there is no predefined way on how to design the “best reward function”.

Issues with the reward function

“Be careful what you wish for - you might get it” (pro-verb)

“...it grants what you ask for, not what you should have asked for or what you intend.” (Norbert Wiener, American mathematician)



Midas and daughter (good as gold) (source: www.flickr.com, by [Robin Hutton](#) CC BY-NC-ND 2.0)

Task-dependent return definitions

Episodic tasks

- A problem which naturally breaks into subsequences (finite horizon).
- Examples: most games, maze,...
- The return becomes a finite sum:

$$g_t = r_{t+1} + r_{t+2} + \cdots + r_T$$

Continuing tasks

- A problem which lacks a natural end (infinite horizon).
- Example: process control task
- The return should be discounted to prevent infinite numbers:

$$\begin{aligned} g_t &= r_{t+1} + \gamma r_{t+2} + \gamma^2 r_{t+3} + \cdots \\ &= r_{t+1} + \gamma(r_{t+2} + \gamma r_{t+3} + \cdots) \\ &= r_{t+1} + \gamma g_{t+1} \end{aligned}$$

Here, $0 \leq \gamma \leq 1$ is the discount rate.

Discounted rewards

$$g_t = r_{t+1} + \gamma r_{t+2} + \gamma^2 r_{t+3} + \dots$$

Numeric viewpoint

- If $\gamma = 1$ and $r_t > 0$ for $k \rightarrow \infty$, g_t becomes infinite.
- If $\gamma < 1$ and r_t is bounded for $k \rightarrow \infty$, g_t is bounded.

Strategic viewpoint

- If $\gamma \approx 1$: agent is farsighted.
- If $\gamma \approx 0$: agent is shortsighted (only interested in immediate reward).

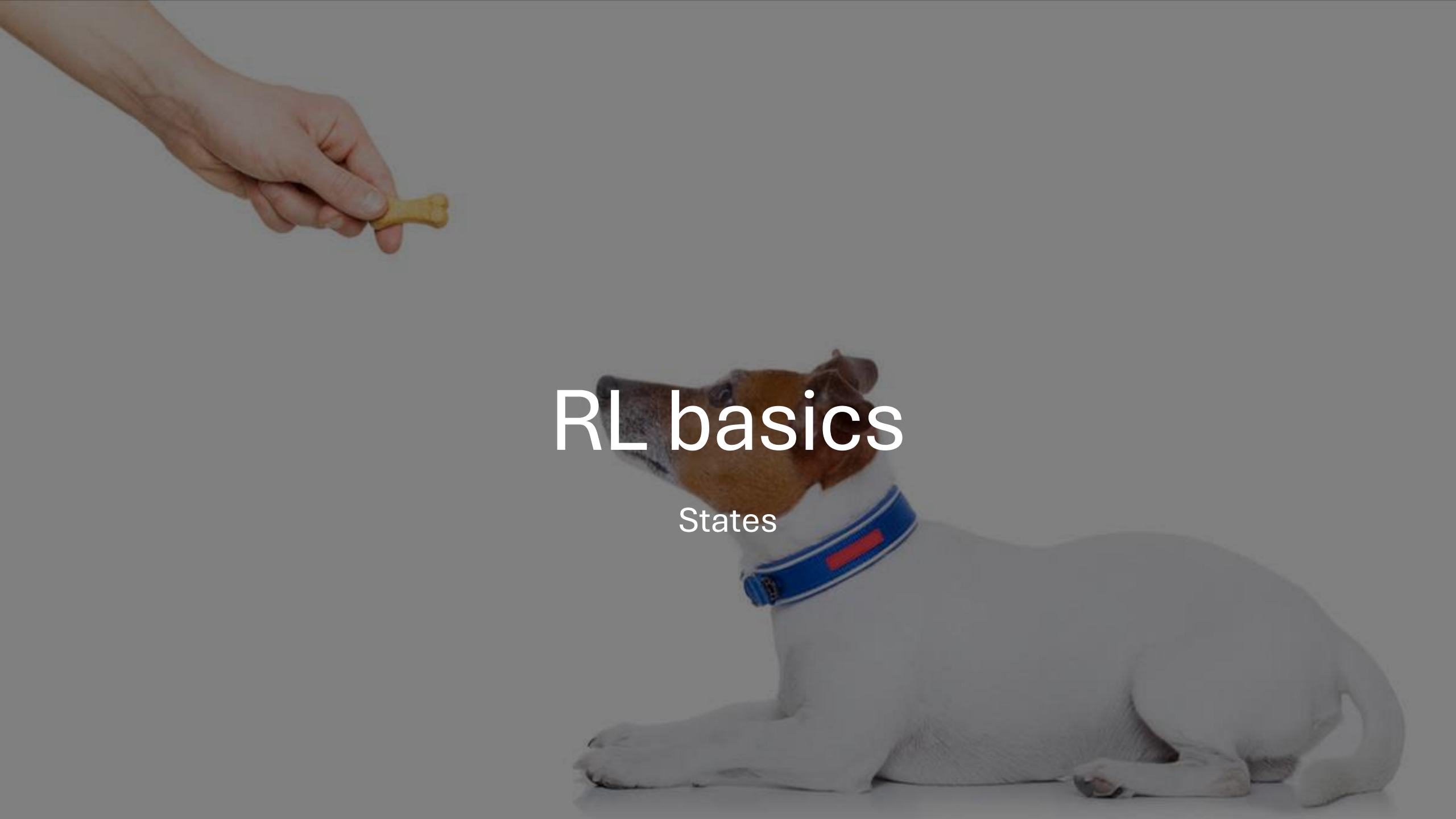
Mathematical viewpoint

- The current return is the discounted future return:

$$g_t = r_{t+1} + \gamma r_{t+2} + \gamma^2 r_{t+3} + \dots = r_{t+1} + \gamma(r_{t+2} + \gamma r_{t+3} + \dots) = r_{t+1} + \gamma g_{t+1}$$

- If $r_t = r$ is a constant and $\gamma < 1$ one receives:

$$g_t = \sum_{i=0}^{\infty} \gamma^i r = r \sum_{i=0}^{\infty} \gamma^i = r \frac{1}{1-\gamma}$$

A hand holding a bone-shaped treat above a dog. The background is a solid light gray. A hand from the top left corner holds a yellow, bone-shaped treat. Below it, a white dog with brown patches on its head and ears is lying down, looking up at the treat. The dog is wearing a blue collar with a red stripe.

RL basics

States

State: Environment state

- Random variable S_t^e with realizations s_t^e
- Internal status representation of the environment, e.g.,
 - Physical states, e.g., car velocity or motor current
 - Game states, e.g., current chess board situation
 - Financial states, e.g., stock market status
- Fully, limited or not at all visible by the agent
 - Sometimes even uncertain
 - In general, the agent may receive a feature vector $\mathbf{x}_t = f(s_t^e)$ as the measurable outputs of the environment
- Continuous or discrete quantity

Notation

Bold symbols are non-scalar multidimensional quantities, e.g., vectors and matrices.

Capitalized symbols denote random variables and lower-case symbols their realizations.

State: Agent state

- Agent construct a state s_t^a from its observations (e.g., from $\mathbf{x}_t$).
- In general: $s_t^a \neq s_t^e$, e.g., due to measurement noise or an additional agent's memory
- The agent state is
 - Agent's condensed information relevant for next action
 - Dependent on internal knowledge / policy representation of the agent
 - Continuous or discrete quantity
- Without specification we will use for the state the agent state:
 $s_t = s_t^a$

History and information state

Definition: The **history** is the past sequence of all observations, actions and rewards

$$H_t = \{s_0, r_0, a_0, \dots, a_{t-1}, s_{t-1}, r_t\}$$

up to the time step t .

If the current state s_t contains all useful information from the history, it is called an information or Markov state (history is fully condensed in s_t)

Definition: A state s_t is called an **information state** if and only if

$$Pr[s_{t+1}|s_t] = Pr[s_{t+1}|s_0, s_1, \dots, s_t]$$

Degree of observability

Full observability

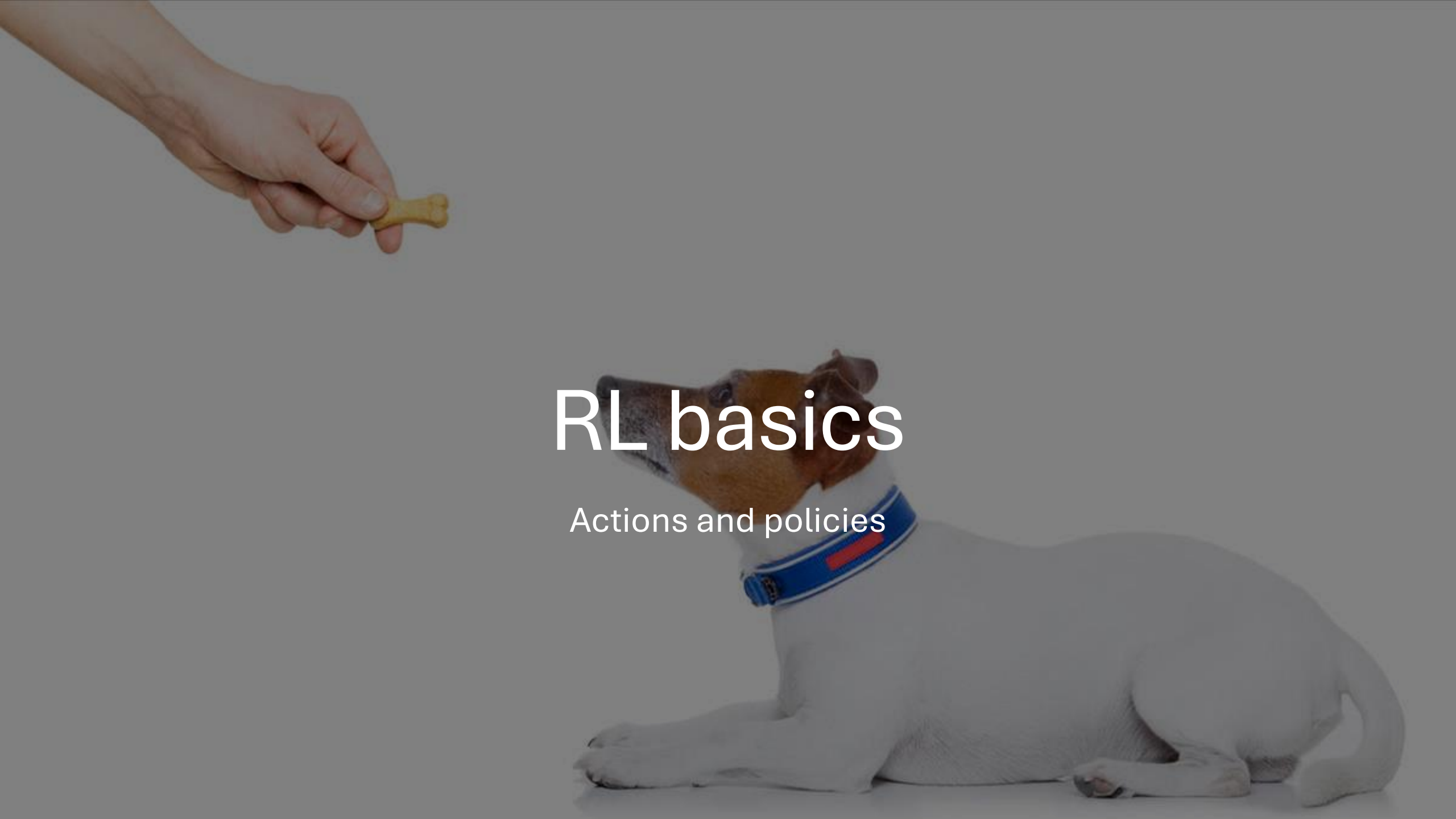
- Agent directly measures full environment state (e.g., $s_t = s_t^e$).
- If s_t^e is an information state: Markov decision process (MDP).

Partial observability

- Agent does not have full access to environment state (e.g., $\mathbf{x}_t = [\mathbf{I} \ 0]\mathbf{x}_t^e$ where $\mathbf{x}_k^e = f(s_k^e)$).
- If s_t^e is Markov: partial observable MDP (POMDP).
- Agent may reconstruct the complete state information $\hat{\mathbf{x}}_t \approx \mathbf{x}_t^e$ (belief, estimate).

POMDP examples

- Technical systems with limited sensors (cutting costs)
- Poker game (unknown opponents' cards)
- Human health status (too complex system)

A hand holding a bone-shaped treat above a dog. The background is a solid light gray. A hand from the top left corner holds a yellow, bone-shaped treat. Below it, a white dog with brown patches on its head and ears is lying down, looking up at the treat. The dog is wearing a blue collar with a red stripe.

RL basics

Actions and policies

Action

- Choosing actions is the agent's mean to maximize its reward.
- Major distinction:
 - Finite action set (FAS): $a_t \in \mathcal{A}$
 - Continuous action set (CAS): Infinite number of actions: $a_t \in \mathbb{R}^m$
 - We use a known (deterministic) action a_t or the random variable A_t
 - Often state-dependent: $a_t \in \mathcal{A}(s_t)$
- Examples:
 - Take a card during a Blackjack game (FAS)
 - Steering while driving an autonomous car (CAS)
 - Buy stock options for your trading portfolio (FAS/CAS)

Policy

- A policy π is the agent's internal strategy on picking actions.

- **Deterministic policies:** maps state and action directly:

$$a_t = \pi(s_t)$$

- **Stochastic policies:** maps a probability of the action given a state:

$$\pi(a_t|s_t) = Pr[a_t|s_t]$$

- RL is all about changing π over time to maximize the expected return.

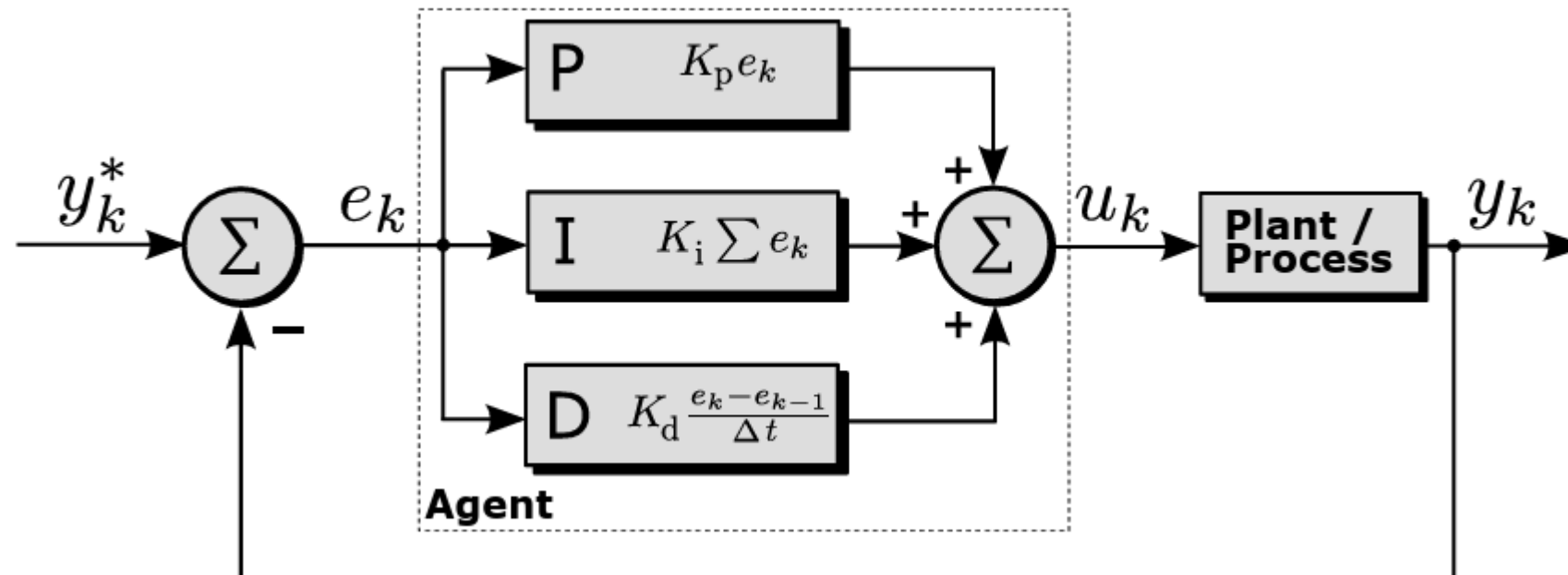
Deterministic policy example

Find optimal gains $\{K_p, K_i, K_d\}$ given the reward $r_k = -e_k^2$:

- Agent's behavior is explicitly determined by $\{K_p, K_i, K_d\}$.
- Reference value is part of the environment state: $\mathbf{x}_k = [y_k \ y_k^*]^T$.
- Control output is the agent's action: $\mathbf{u}_k = \boldsymbol{\pi}(\mathbf{x}_k | K_p, K_i, K_d)$.

Control theory notation:

- $\mathbf{x}_k$ is here the state at time k as a feature vector.
- $\mathbf{u}_k$ is the action
- $\mathbf{y}_k$ is the observation



Classical PID control loop with scalar quantities (derivative of www.wikipedia.org, by Arturo Urquiza [CC BY-SA 3.0](https://creativecommons.org/licenses/by-sa/3.0/))

Stochastic policy

- Two-player game of extended rock-paper-scissors:
- A deterministic policy can be easily exploited by the opponent.
- A uniform random policy would be instead unpredictable (assuming an ideal random number generator).

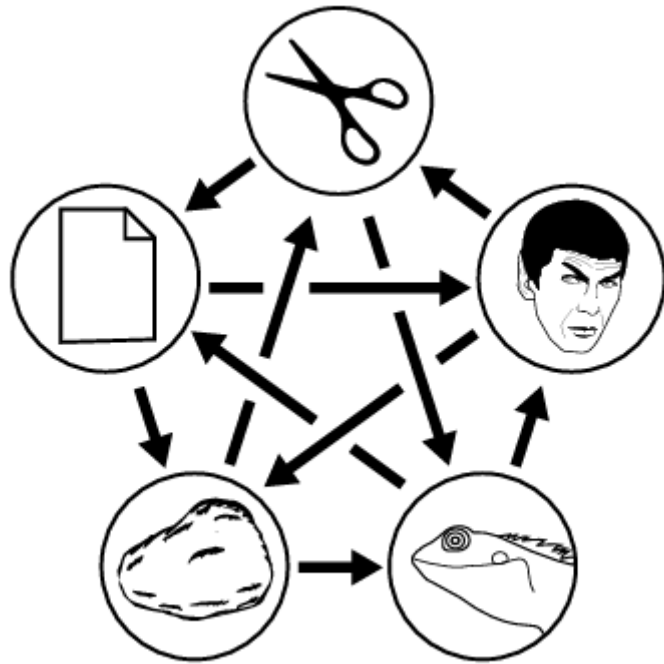


Figure: Rock paper scissors lizard Spock game mechanics (source: www.wikipedia.org, by Director Doc [CC BY-SA 4.0](https://creativecommons.org/licenses/by-sa/4.0/))

Value Function

- The value function estimates the **long-term rewards**.
- Basis for choosing actions.
- Defined as the expected return being in state s_t following π : $v_\pi(s_t)$.

$$v_\pi(s_t) = \mathbb{E}_\pi[G_k | S_t = s_t] = \mathbb{E}_\pi \left[\sum_{i=0}^{\infty} \gamma^i R_{t+i+1} \middle| s_t \right]$$

- The action-value function is the expected return being in state s_t taken an action a_t and, thereafter, following a policy

$$q_\pi(s_t, a_t) = \mathbb{E}_\pi[G_k | S_t = s_t, A_t = a_t] = \mathbb{E}_\pi \left[\sum_{i=0}^{\infty} \gamma^i R_{t+i+1} \middle| s_t, a_t \right]$$

- A key task in RL is to estimate $v_\pi(s_t)$ and $q_\pi(s_t, a_t)$ based on sampled data.

Exploration and exploitation

- In RL the environment is initially unknown.
- The agent needs to learn to act optimal. That is to use the reward signal to alter its behavior policy.
- **Exploration:** find out more about the environment.
- **Exploitation:** maximize current reward using limited information.
- **Trade-off problem:** what's the best split between both strategies?

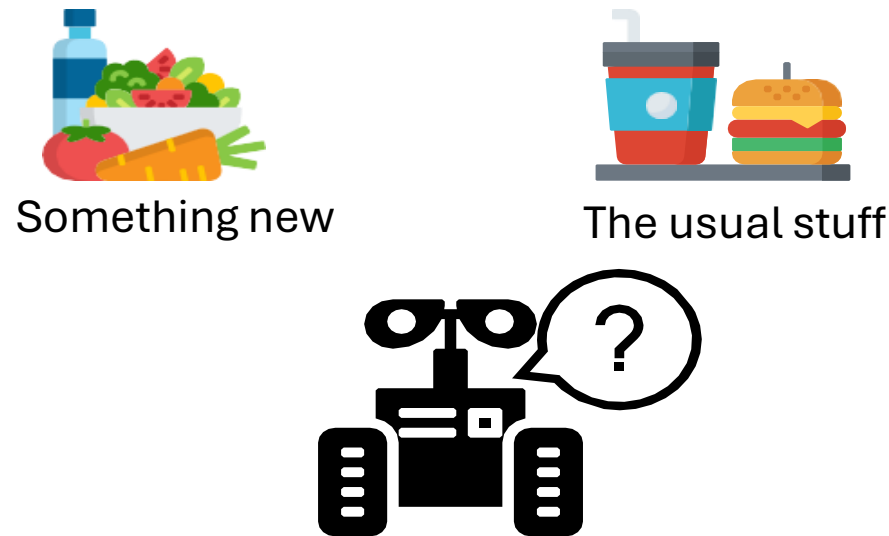


Figure: The exploration-exploitation dilemma

Model

- Sometimes we have a model of the environment.
- A model predicts what will happen inside an environment.
- A model consists of two parts:

- State transition model $\mathcal{P}$:

$$\mathcal{P} = \Pr[S_{t+1} = s_{t+1} | S_t = s_t, A_t = a_t]$$

- Reward model $\mathcal{R}$:

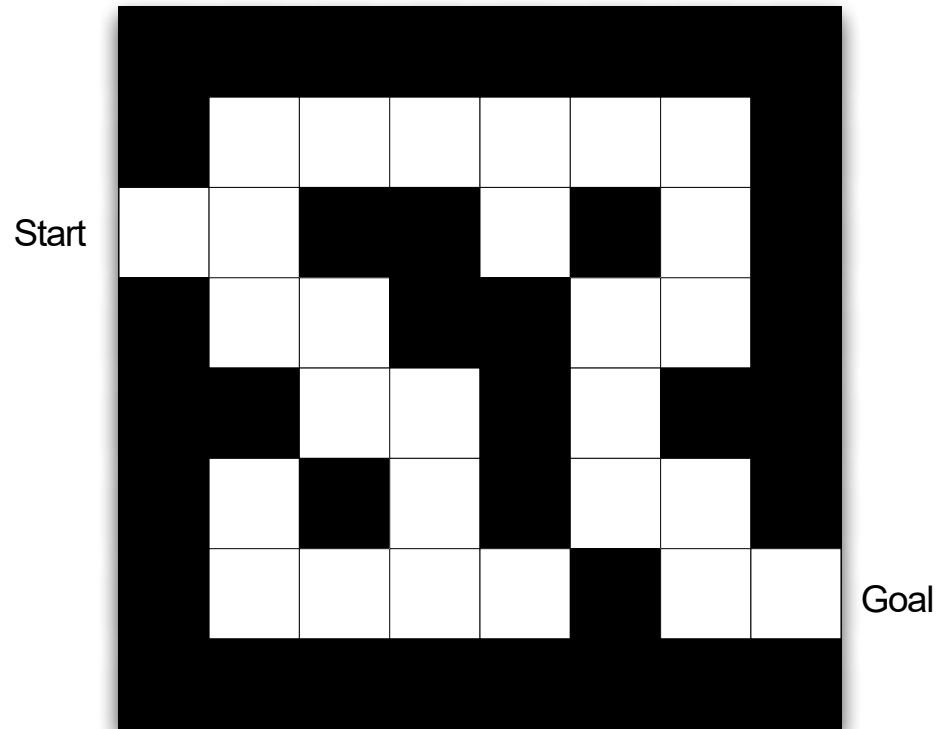
$$\mathcal{R} = \Pr[R_{t+1} = r_{t+1} | S_t = s_t, A_t = a_t]$$

- In general, those models could be stochastic (as denoted above) but in some problems relax to a deterministic form.
- Using data in order to fit a model is a learning problem of its own and often called system identification.

A small example



Maze example



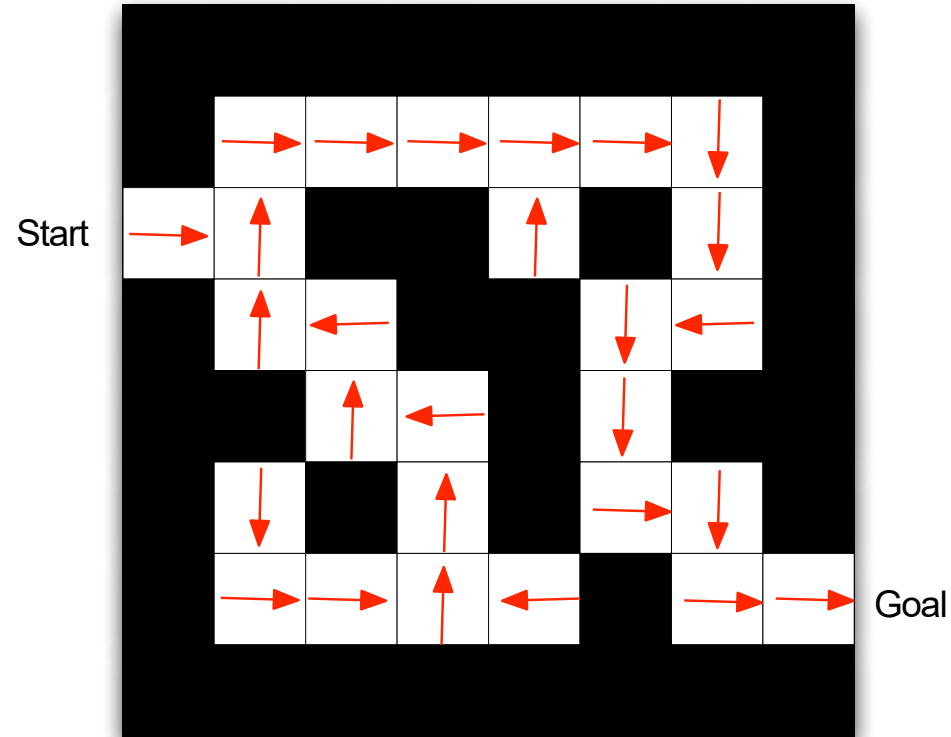
Maze setup

Problem statement

- Reward: $r_t = -1$
- At goal: episode termination
- Actions: $a_t \in \{N, E, S, W\}$
- State: agent's location
- Deterministic problem

(source: D. Silver, "Reinforcement learning", 2015. [CC BY-NC 4.0](#))

Maze example: RL-solution by policy



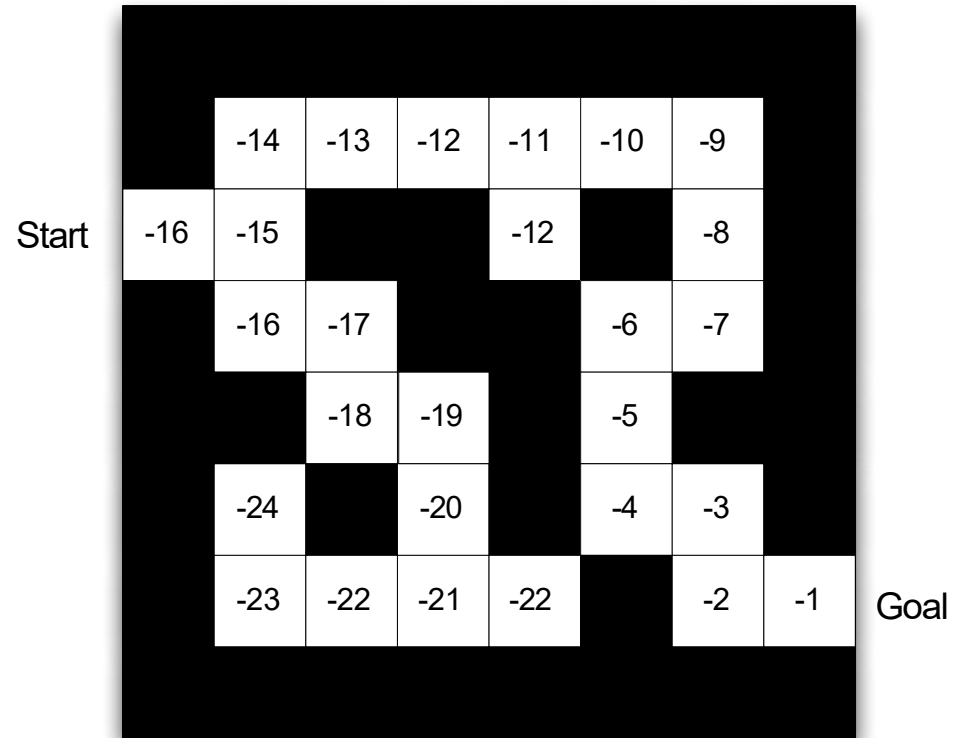
Maze setup: Arrows represent policy $\pi(s_t)$

(source: D. Silver, “Reinforcement learning”, 2015. [CC BY-NC 4.0](https://creativecommons.org/licenses/by-nc/4.0/))

Key characteristics

- For any state there is a direct action command.
- The policy is explicitly available.

Maze example: RL-solution by value function



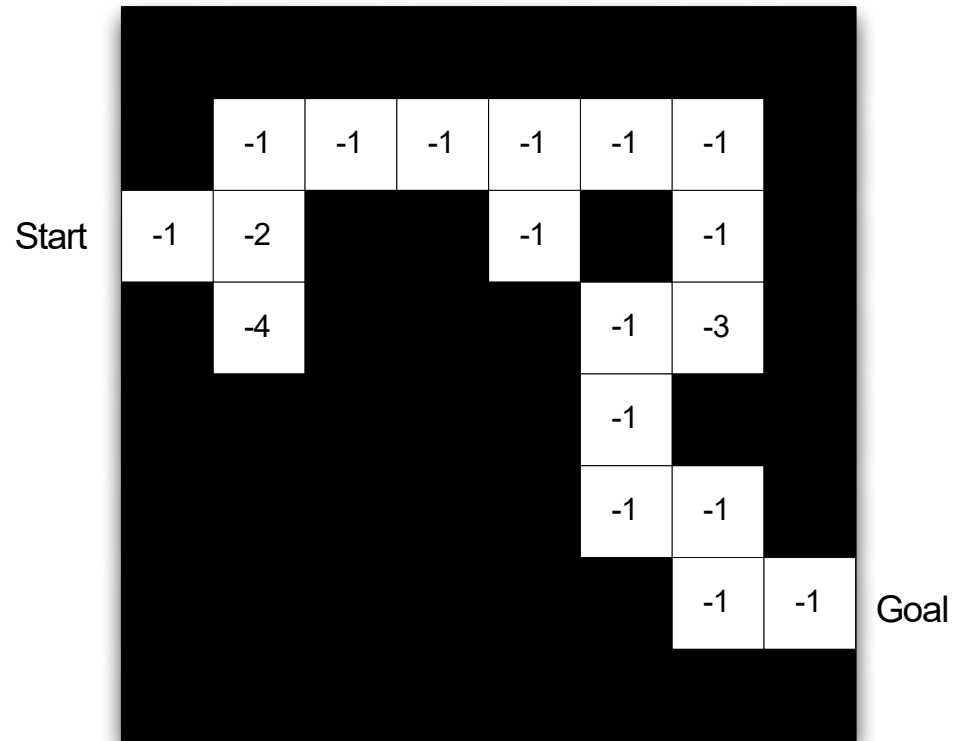
Numbers represent value $v_{\pi}(s_t)$

(source: D. Silver, “Reinforcement learning”, 2015. [CC BY-NC 4.0](https://creativecommons.org/licenses/by-nc/4.0/))

Key characteristics

- The agent evaluates neighboring maze positions by their value. Goal
- The policy is only implicitly available.

Maze example: RL-solution by model evaluation



Key characteristics

- Agent uses internal model of the environment.
- The model is only an estimate (inaccurate, incomplete).
- The agent interacts with the model before taking the next action (e.g., by numerical optimizers).

Grid layout represents the explored part of the state transition model $\mathcal{P}$

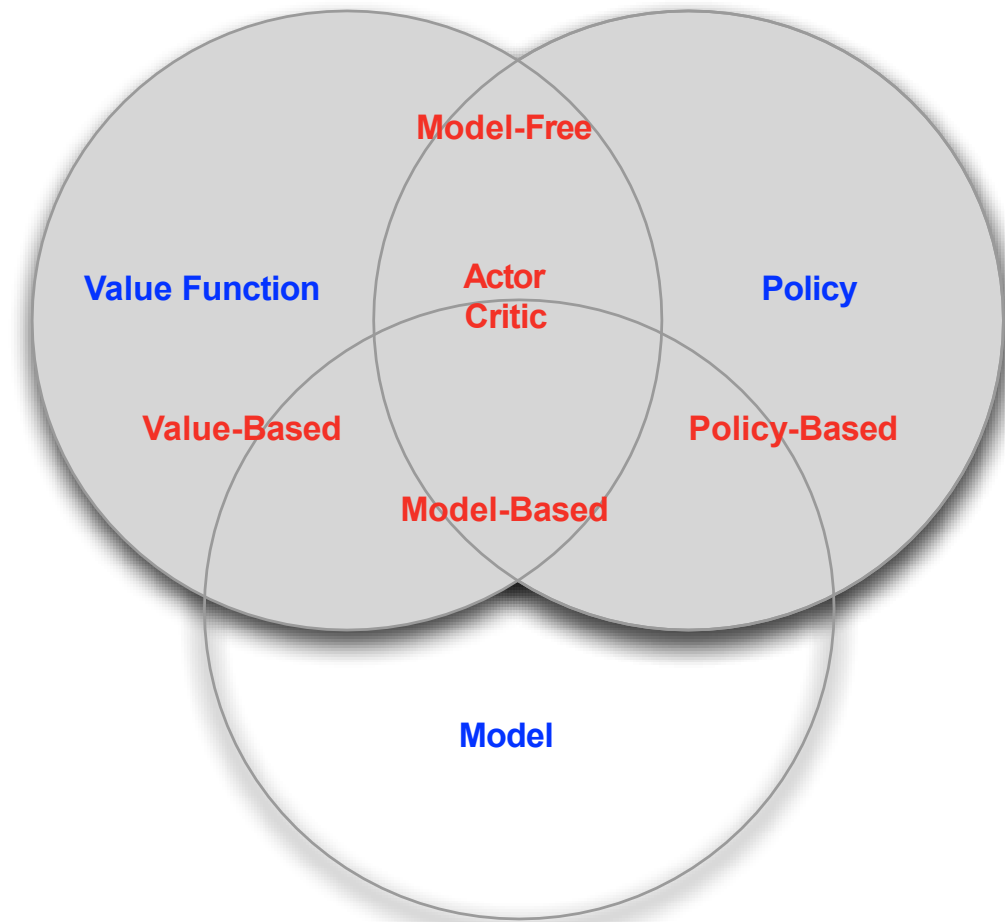
and numbers depict the estimate by the reward model $\mathcal{R}$

(source: D. Silver, "Reinforcement learning", 2015. [CC BY-NC 4.0](#))

A hand holding a bone over a dog. A hand is visible in the upper left corner, holding a small, yellow, bone-shaped object. Below the hand, a dog is lying down, looking up at the bone. The dog has white fur with brown patches on its head and ears. It is wearing a blue collar with a red stripe. The background is a solid light gray.

RL and planning

RL agent taxonomy



Main categories of reinforcement learning algorithms
(source: D. Silver, "Reinforcement learning", 2015. [CC BY-NC 4.0](#))

RL vs. planning

Two fundamental solutions to sequential decision making:

- Reinforcement learning:
 - The environment is initially unknown.
 - The agents interacts with the environment.
 - The policy is improved based on environment feedback (reward).
- Planning:
 - An a priori environment model exists.
 - The agents interacts with its own model.
 - The policy is improved based on the model feedback ('virtual reward').
- Remark on learning and planning
- Above the two extreme cases are confronted:
 - RL = learning based on data obtained from interacting with the system.
 - Planning = iterating on a model without improving it based on data.
- Can one of these extreme cases lead alone to an efficient and optimal solution?



Summary: what you should know

- Understanding the role of RL in machine learning and optimal sequential decision making.
- Become acquainted with the basic RL interaction loop (agent, environment, interpreter).
- Finding your way around the basic RL vocabulary.
- Internalize the significance of proper reward formulations (design parameter).
- Differentiate solution ideas on how to retrieve an optimal agent behavior (policy).
- Delimit RL towards model predictive control as a sequential decision-making alternative.